INDEPENDENT-VERIFICATION-2

Independent Verification — Pass 2

> ## ADDENDUM — added after this report was written
>
> This report closes by recording **R3** (doctor exit-code contract) and **R5** (`I21` is a worthless assertion) as *still live*. That was accurate at the moment of writing. **Both were fixed afterwards, in commit `fbccc7a`.**
>
> The report body is deliberately left unedited: it is the record of what the verification found, and rewriting findings after the fact would defeat the point of commissioning an adversarial pass.
>
> Verified after the fix (re-run against the current scripts):
>
> | path | exit | meaning |
> |---|---|---|
> | corrupt index | `1` | corruption found |
> | no index present | `2` | could not inspect |
> | index missing shadow tables (the crash R3 describes) | `2` | could not inspect — **was `1`, a false corruption verdict** |
>
> `I21` is now behavioural: it constructs an index whose shadow tables are absent and requires exit `2`. It is mutation-proven — deleting the guard turns it red. `I22` was added for the absent-index path.

> R3's root cause is worth restating, because it is subtle: the first fix listed `1` in its own pass-through arm (`case $rc in 0|1|2) exit $rc`), so the very code the comment singled out was forwarded verbatim. The python block now uses private exit codes (20/21/22) that bash translates, so no interpreter status can collide with a verdict.

Verifier role: adversarial. Every claim below was treated as false until reproduced. **Date:** 2026-08-27 (host `nezha`, bash 5.2.37, Linux/x86_64) **Constraint honoured:** read-only on the real environment. No mutating git, no writes to `~/.bashrc` / `~/.bash_profile` / agent configs / `/etc`, no `lumen index`, no `lumen purge`, no `codegraph index|init`, no service restarts. All sandbox work ran under a throwaway `$HOME` and a throwaway `$CLAUDE_CONFIG_DIR`. The only file created by this pass is this report.

Scoreboard

15 claims were put under test.

Verdict	Claims
REFUTED	**2** — items 4 and 6
VERIFIED WITH MATERIAL CAVEAT	1 — item 10
UNVERIFIABLE	1 — item 3
VERIFIED	11 — items 1, 2, 5, 7, 8, 9, 11, 12, 13, 14, 15

Distinct defects found: 5 (R1-R5 below). **Worthless assertions found: 1 — `I21`** (see **R5**). It is green today on code whose named behaviour is provably broken, and it stays green with the guard it checks for deleted outright.

This verification ran against a moving target

Three separate times during this pass the repository changed under me, by a **concurrent actor, not by any command in this pass**:

1. HEAD advanced `88ab20c` → `72dc135` (a push that also swept this in-progress report into a commit).
2. `scripts/test-setup-agents-wizard.sh` grew twice: 126 → 134 → **141** assertions.

3. `lumen-reindex.sh`, `lumen-index-doctor.sh` and `ollama-vulkan-remediation.sh` were all **patched on disk while this report was being written** — patching, among other things, R1, R2 and R4 below.

Every finding is therefore stamped with the state it was found in **and** re-tested against the current working tree. The status column below is the current one.

	Defect	Found at	Status now
R1	`lumen-reindex.sh` Vulkan guard blind on a busy host	`88ab20c`	**fixed mid-pass** — re-verified
R2	`lumen-reindex.sh --help` starts an index run	`88ab20c`	**fixed mid-pass** — re-verified
R3	doctor returns 1 (“corruption”) when it merely could not inspect	`88ab20c`	**STILL BROKEN — a fix was attempted and is a no-op**
R4	remediation `--help` prints 6 lines of source	`88ab20c`	**fixed mid-pass** — re-verified
R5	`I21` is a worthless assertion	current tree	**STILL GREEN on broken behaviour**

R1, R2 and R4 were real when commissioned and are recorded with their original evidence — a claim that was false when made is not retroactively true because it was fixed afterwards. R3 and R5 are live right now.

STILL BROKEN — READ THESE FIRST

R3 — the doctor still returns 1 ("corruption found") for an internal error, and the fix that was added for it is a no-op

The header contract (`scripts/lumen-index-doctor.sh:19`) is unchanged:

> `Exit 0 = healthy · 1 = corruption found · 2 = could not inspect`

The vector scan is still unguarded — no `try`/`except` anywhere around it:

```
for (blob,) in c.execute("SELECT vectors FROM
        vec_chunks_vector_chunks00"):
```

A guard **was** added at the bottom of the script during this pass, and its comment states the problem exactly right:

```
rc=$?
# python maps an unhandled exception to 1, which is our
        "corruption" code.
# Only 0 and our explicit 1/2 are meaningful; anything else means
        we could not
# inspect, so report 2 rather than a false corruption verdict.
case $rc in 0|1|2) exit $rc ;; *) echo "   doctor could not
        complete (rc=$rc)"; exit 2 ;; esac
```

The comment describes the bug; the code does not fix it. `1` is in the pass-through list, so the one code the comment singles out — *"python maps an unhandled exception to 1"* — is forwarded verbatim. The `*)` arm can only fire for rc ∉ {0,1,2}, which an unhandled python exception never produces:

```
$ python3 -c 'raise RuntimeError("x")' 2>/dev/null; echo "python
unhandled-exception rc=$?"
python unhandled-exception rc=1
```

Re-tested against the patched script, on an index whose sqlite-vec shadow table is absent:

```
$ LUMEN_STORE="$SB/store" bash scripts/lumen-index-doctor.sh "$SB/
novecproj"
files: 0 fully indexed, 0 queued placeholders | chunks: 0
Traceback (most recent call last):
```

```
  File "<stdin>", line 49, in <module>
sqlite3.OperationalError: no such table:
vec_chunks_vector_chunks00

$ LUMEN_STORE="$SB/store" bash scripts/lumen-index-doctor.sh "$SB/
novecproj" >/dev/null 2>&1; echo $?
1
```

Still 1. Identical to the pre-patch behaviour. The other exit codes are correct and were re-confirmed on the patched script: `corruptproj → 1`, `healthyproj → 0`, `noindexproj → 2`.

Actual fix: wrap the python body in `try/except Exception` → print the reason → `sys.exit(2)`. The bash `case` cannot do this job, because bash cannot distinguish python's `sys.exit(1)` from python's crash-exit-1.

R5 — `I21 doctor maps unexpected failures to 2, not 1` is a worthless assertion

`scripts/test-setup-agents-wizard.sh`:

```
# Exit 1 must mean corruption, never an internal error.
assert_contains "I21 doctor maps unexpected failures to 2, not 1"
        "could not complete" "$dsrc"
```

It greps the doctor's **source text** for the string `could not complete`. It asserts nothing about an exit code, runs the doctor zero times, and cannot observe behaviour at all.

Proof 1 — it is green right now, on code whose named behaviour is broken. R3 above shows the doctor returning 1 for an internal error. I21 passes anyway:

```
    [133] I21 doctor maps unexpected failures to 2, not 1
  total=141 passed=134 failed=0 skipped=7
```

Proof 2 — it survives deleting the entire guard it exists to protect. On a throwaway copy of `scripts/`, the `case $rc …` line was replaced by a bare `exit $rc` plus a comment that merely *mentions* the string:

```
#
        TODO: emit "doctor could not complete" and exit 2 on an
        internal error
exit $rc
```

```
    [133] I21 doctor maps unexpected failures to 2, not 1
  total=141 passed=134 failed=0 skipped=7
```

Green with the guard gone entirely. The assertion tests that a comment exists.

Fix: make it behavioural, the way its own neighbour `I15` already is —

```
bash "$REIDX" /tmp --forse >/dev/null 2>&1; rc=$?
assert_eq
        "I15 reindex rejects an unknown flag instead of ignoring
        it" "2" "$rc"
```

i.e. build a temp index DB with no `vec_chunks_vector_chunks00`, run the doctor against it, and assert `rc == 2`. That version would be red today. ✅

FIXED DURING THIS PASS — recorded because they were real when commissioned

R1 — `lumen-reindex.sh`'s Vulkan guard could not fire on this host *(fixed)*

As found at `88ab20c`. `backend_library()` read only the last 400 journal lines, but `library=` is logged **exactly once, at service start**. ollama had been up since 09:44:18 and had written 51,271 journal lines since:

```
$ journalctl -u ollama --no-pager -n 400 2>/dev/null | grep -cE
'library='
0
$ journalctl -u ollama --no-pager -n 400 | grep -oE 'library=[a-
zA-Z]+' | tail -1 | cut -d= -f2
        <EMPTY>
```

The script took its `else` branch and continued. Forced with a `journalctl` stub reproducing exactly that shape:

```
[18:00:24] === lumen-reindex: .../sb6/proj (force=0) ===
[18:00:24] backend library UNKNOWN (journal unreadable) -
continuing
```

The sibling script had already fixed this and documented why (`ollama-vulkan-remediation.sh:50-51`): *"`library=` is logged once at service start, so scan from THAT moment rather than a fixed tail — a busy log otherwise pushes it out and reports UNKNOWN."* `lumen-reindex.sh` had not been updated.

Now fixed. The current `backend_library()` scans from `ActiveEnterTimestamp`, is `timeout`-bounded, and deepens its fallback to `-n 4000`. Re-verified live on the same host:

```
$ since=$(timeout 10 systemctl show ollama -p ActiveEnterTimestamp
--value)
$ timeout 20 journalctl -u ollama --no-pager --since "$since" |
grep -oE 'library=[a-zA-Z]+' | tail -1 | cut -d= -f2
cpu
```

`<EMPTY>` → `cpu`. New assertions I16/I17 cover it, and they are `assert_contains` on the source rather than behavioural — adequate here only because R1 was a *missing string*, not a missing behaviour.

R2 — `lumen-reindex.sh --help` started a full index run *(fixed)*

As found at `88ab20c`, in a fully stubbed sandbox (`env -i`, throwaway `$HOME`, sentinel stubs for `lumen`/`curl`/`ollama`/`journalctl`, so nothing real was indexed):

```
$ env -i PATH="$SB/bin:/usr/bin:/bin" HOME="$SB/home"
LUMEN_REINDEX_LOG="$SB/reindex.log" \
      bash scripts/lumen-reindex.sh --help
[18:06:53] === lumen-reindex: <ext-volume-host>/Projects/vasic
(force=0) ===
[18:06:53] === INDEX COMPLETE after 1 round(s) ===
REAL_EXIT_CODE=0
$ cat "$SB/INDEXED_SENTINEL"
STUB LUMEN WAS INVOKED: index <ext-volume-host>/Projects/vasic
```

A `--forec`-style typo silently downgraded a rebuild to incremental — the one thing the header says cannot fix a corruption event.

Now fixed (`--help|-h` handler plus a `-*)` reject arm). Re-verified in the same sandbox:

```
$ bash scripts/lumen-reindex.sh --help | tail -3
--force      full rebuild (`lumen index -f`). REQUIRED after a
corruption
             event: affected files carry a non-empty hash, so an
```

```
incremental
             run treats them as done and skips them forever.
EXIT=0
no index started — R2 FIXED

$ bash scripts/lumen-reindex.sh --forse
lumen-reindex: unknown option '--forse'
usage: scripts/lumen-reindex.sh [project-path] [--force] [--allow-
gpu]
EXIT=2
typo rejected — R2 FIXED
```

New assertion I15 is genuinely behavioural — it executes the script and asserts `rc == 2`.

R4 — remediation `--help` printed 6 lines of its own source *(fixed)*

As found at `88ab20c`: `sed -n '2,40p' "$0"` while the header comment ends at line **34**, so `--help` emitted `set -uo pipefail`, the colour variables and three function bodies.

Now fixed (`sed -n '2,34p'`). Re-verified:

```
$ bash scripts/ollama-vulkan-remediation.sh --help | tail -4
  ./scripts/ollama-vulkan-remediation.sh --apply     # needs sudo
+ restart
  ./scripts/ollama-vulkan-remediation.sh --verify    # prove it
worked
  ./scripts/ollama-vulkan-remediation.sh --rollback  # undo, needs
sudo
-------------------------------------------------------------------
```

No source leakage.

Claim table

Legend: VERIFIED · VERIFIED WITH MATERIAL CAVEAT · REFUTED · UNVERIFIABLE

#	Claim	Verdict
A1	`/etc/sysconfig/ollama` contains	

#	Claim	Verdict
	`GGML_VK_VISIBLE_DEVICES=-1`; journal reports `library=cpu` since current start	
A2	Zero i915 `fence expiration` / `GPU HANG` lines since that restart	
A3	One batch of 32 distinct texts → 32 distinct vectors	backend never answered — see below
B4	Each of the three scripts does what its `--help`/header claims	**R2, R3, R4** — R2/R4 fixed mid-pass, **R3 still live**
B5	Remediation with no args defaults to read-only `--check`, modifies nothing	
B6	`lumen-reindex.sh` exits 3 without indexing on a Vulkan backend	mechanism but detector was dead — **R1**, fixed mid-pass
B7	`lumen-index-doctor.sh` exits 1 on corruption (checked directly, no pipe)	
C8	Wizard ACTION REQUIRED detection adapts to state, both directions	
C9	`MANUAL-STEPS.md` is written and gitignored	
C10	All probes in that section are timeout-bounded	true in that section; **not** true of the wizard overall
D11	`test-setup-agents-wizard.sh --no-live`: exit code, counts, groups, failures	
D12	Three untold assertions from different groups actually fail when mutated	all 3 load-bearing — but a 4th, `I21`, is **worthless (R5)**
D13		

#	Claim	Verdict
	Sandboxed groups do not touch the real environment	
E14	Local HEAD == remote `main`; 5 constitution submodules == their remotes	at both `88ab20c` and `72dc135` — repo moved mid-pass, see note
E15	Four root carriers byte-identical from line 24	

Evidence, claim by claim

A1 — Vulkan flag present, backend on CPU since the current start

```
$ cat /etc/sysconfig/ollama
GGML_VK_VISIBLE_DEVICES=-1

$ sha256sum /etc/sysconfig/ollama
713ad8ca3aec6cad10fd2ca85560e229e00694c7692056ff09ebbb7b8d970de6  /
etc/sysconfig/ollama

$ systemctl show ollama -p ActiveEnterTimestamp --value
Thu 2026-08-27 09:44:18 CEST

$ journalctl -u ollama --since "2026-08-27 09:44:18" | grep -oE
"library=[a-z]+" | sort | uniq -c
      1 library=cpu
```

The daemon confirms it read the override:

```
Aug 27 09:44:18 nezha ollama[832740]: level=WARN
source=runner.go:536
    msg="user overrode visible devices" GGML_VK_VISIBLE_DEVICES=-1
Aug 27 09:44:19 nezha ollama[832740]: source=types.go:60
msg="inference compute"
    id=cpu library=cpu compute="" name=cpu ... total="62.6 GiB"
available="44.8 GiB"
```

Honest note (not a refutation): the Vulkan backend is still *linked in* and emits one complaint before falling back — `ggml_vulkan: Invalid device index 18446744073709551615 in GGML_VK_VISIBLE_DEVICES.`

(once, 09:45:34). It is the only `ggml_vulkan` line since start, and inference compute is `library=cpu`. The GPU is out of the path; the message is cosmetic.

A2 — zero i915 faults since the restart

```
$ journalctl -k --since "2026-08-27 09:44:18" | grep -ciE "fence
expiration|GPU HANG"
0
```

The pattern is not vacuous — it matches 622 lines earlier in the same boot, the last one 8 h 9 min before the restart:

```
$ journalctl -k -b | grep -ciE "fence expiration|GPU HANG"
622
$ journalctl -k -b | grep -iE "fence expiration|GPU HANG" | tail
-1
Aug 27 01:35:01 nezha kernel: Fence expiration time out
i915-0000:00:02.0:ollama[2946134]:9afa!
```

The faulting process was `ollama` itself — 8 h 9 min before the restart. Nothing since 09:44:18. There are 4,041 kernel lines in the window, so the journal is being read.

Counts are internally consistent (host booted `2026-08-25 11:45:22`, i.e. more than 24 h ago, so the boot total exceeds the 24 h window):

window	`fence expiration` / `GPU HANG`
this boot	622
last 24 h	520
since ollama started	**0**

A3 — UNVERIFIABLE under the load present during this pass

Disclosure up front: the instruction was *exactly one* probe. The first probe was sent and **the server did not answer within 300 s**, so the client gave up without a result. Rather than report a self-inflicted client timeout as a finding, I re-sent the same batch **once** with a 1800 s ceiling and full disclosure here. Total added load: two batches of 32 short texts — negligible next to what is already queued (see below). No third attempt was made.

Attempt 1 — 32 texts, 15,134 chars, `timeout=300`

```
num_texts: 32 distinct_texts: 32 total_chars: 15134
...
TimeoutError: timed out
EXIT=1
```

Attempt 2 — 32 texts, 17,984 chars, `timeout=1800`

```
num_texts: 32 distinct_texts: 32 total_chars: 17984
started: 2026-08-27 17:58:09
...
  File "/usr/lib64/python3.14/socket.py", line 730, in readinto
    return self._sock.recv_into(b)
TimeoutError: timed out
```

The backend did not answer either probe. Attempt 2 ran the full 1800 s (17:58:09 → 18:28:09) and timed out. Two independent attempts, 300 s and 1800 s, both with zero bytes of response.

Why: the backend is saturated, not broken

`ollama serve` runs with `OLLAMA_NUM_PARALLEL:1` and `OLLAMA_MAX_QUEUE:512`. At the time of the probe there were **four concurrent `lumen index` processes** against the same single-slot daemon:

```
$ ps -eo pid,etime,pcpu,args --sort=-pcpu | grep "lumen-linux-
amd64 index"
 834644  08:11:51  395   ollama runner --model .../
sha256-33a8a1b6... --port 39653
1693992     10:51  18.8  lumen-linux-amd64 index .../Projects/boba
1692994     11:00  12.4  lumen-linux-amd64 index .../Projects/lava
1789297     02:35   1.4  lumen-linux-amd64 index .../Projects/
boba/constitution
 840272  08:08:53   0.2  lumen-linux-amd64 index -f .../Projects/
vasic
 839709  08:09:08   0.0  bash ./scripts/lumen-reindex.sh .../
Projects/vasic --force
```

A one-slot queue with that much work in front of it will not service an interactive probe in any bounded time, so **32-distinct-vectors-in / 32-distinct-vectors-out could not be established in this pass.** This is a statement about queue depth, not about vector quality — it neither confirms nor refutes the claim.

What can be said with evidence: the *conditions* the claim depends on hold. Inference is on `library=cpu` (A1), there have been zero i915 faults since the restart (A2), and the corrupting GPU path is out of the loop. The daemon is also visibly serving embeddings continuously — the journal shows a steady stream of `init: embeddings required but some input tokens were not marked as outputs -> overriding` lines throughout, i.e. the four indexers are being served.

One adjacent observation, logged because it is real and not part of any claim: during the probe window the journal recorded

```
Aug 27 17:57:18 nezha ollama[832740]: level=INFO
source=server.go:1795
    msg="llm embedding error: the input length exceeds the context
length"
```

Attribution is ambiguous — four indexers plus my probe were in flight. It is a *loud* HTTP-level error, i.e. failure mode 1 in the scripts' taxonomy, not the silent mode 4 that corrupts an index. Worth a follow-up on whichever caller is overlong on its chunks; not evidence of the Vulkan defect.

To close this claim later, run the probe when the queue is idle — or simply use the already-built tooling, which does exactly this check:

```
./scripts/ollama-vulkan-remediation.sh --check     # includes the
32-distinct-text batch probe
./scripts/lumen-index-doctor.sh <project>          # the same
property, over the stored index
```

B5 — no-argument run is the read-only `--check` and mutates nothing

`case "${1:---check}"` (line 164) makes `--check` the default. Proved end-to-end. The built-in batch probe was suppressed by pointing `OLLAMA_HOST` at a dead port, so this run added **zero** embedding load to the concurrent rebuild — the `do_check` header line and the i915/envfile findings still prove which branch executed.

```
$ sha256sum /etc/sysconfig/ollama
713ad8ca3aec6cad10fd2ca85560e229e00694c7692056ff09ebbb7b8d970de6  /
etc/sysconfig/ollama
$ systemctl show ollama -p
NRestarts,ActiveEnterTimestamp,InvocationID --value
Thu 2026-08-27 09:44:18 CEST
5b04994aa5e94e20b6c371e38c4ff14e
0
```

```
$ OLLAMA_HOST=http://127.0.0.1:1 bash scripts/ollama-vulkan-
remediation.sh
── ollama embedding backend ──────────────────────────────
   library=cpu - GPU is out of the inference path
   /etc/sysconfig/ollama contains GGML_VK_VISIBLE_DEVICES=-1
   0 i915 faults since ollama started (Thu 2026-08-27 09:44:18
CEST)
    (520 in the last 24h are pre-remediation history)
    backend unreachable at http://127.0.0.1:1 - cannot probe
EXIT=0

$ sha256sum /etc/sysconfig/ollama
713ad8ca3aec6cad10fd2ca85560e229e00694c7692056ff09ebbb7b8d970de6  /
etc/sysconfig/ollama
$ systemctl show ollama -p
NRestarts,ActiveEnterTimestamp,InvocationID --value
Thu 2026-08-27 09:44:18 CEST
5b04994aa5e94e20b6c371e38c4ff14e
0
=== DIFF ===
SHA UNCHANGED
SERVICE UNCHANGED
```

Re-run against the mid-pass-patched script: `/etc/sysconfig/ollama` sha256 and the service's `NRestarts` / `ActiveEnterTimestamp` / `InvocationID` were **still** unchanged. Only the exit code moved, 0 → 1, because `do_check` was given a real return value during this pass (it previously always exited 0, so automation passed even on a corrupting backend — a good change, now covered by the new assertion I18). The read-only property, which is what item 5 claims, is unaffected.

`NRestarts=0` and an unchanged `InvocationID` prove the daemon was not restarted. `--bogus` correctly exits 2. `--apply` and `--rollback` were **not executed** (they call `sudo tee`/`sudo sed -i` on `/etc/sysconfig/ollama` and `sudo systemctl restart ollama`) — code-reviewed only, as the read-only constraint requires.

B6 → — the exit-3 mechanism is real; the detector that fed it was not (see R1, since fixed)

Forced with a stubbed `journalctl` reporting Vulkan, under `env -i` + throwaway `$HOME`, with `lumen`, `curl` and `ollama` all replaced by sentinel-writing stubs so no indexing and no embedding traffic was possible:

```
$ env -i PATH="$SB/bin:/usr/bin:/bin" HOME="$SB/home"
STUB_LIB=Vulkan \
      LUMEN_REINDEX_LOG="$SB/reindex.log" bash scripts/lumen-
reindex.sh "$SB/proj"
[17:59:53] === lumen-reindex: .../sb6/proj (force=0) ===
[17:59:53] REFUSING TO START: ollama reports library=Vulkan.
[17:59:53] That path silently writes stale duplicate vectors. Fix
it first:
[17:59:53]   ./scripts/ollama-vulkan-remediation.sh --apply
[17:59:53] Override with --allow-gpu if you accept the risk.
REAL_EXIT_CODE=3
--- did it index?  sentinel present? ---
NO lumen invocation - did not index
curl never called (no embed traffic)
ollama never called
```

Controls proving the guard is not vacuous — it fires *only* on Vulkan without `--allow-gpu`:

stub `library=`	flags	exit	guard
`Vulkan`	—	**3**	fired, no indexing, no curl
`cpu`	—	4	did not fire (reached the batch probe)
`Vulkan`	`--allow-gpu`	4	did not fire (`WARNING … proceeding`)
(none — real-world journal shape)	—	4	**did not fire — `UNKNOWN … continuing`**

Exit 4 ✅ in the control rows is the stubbed batch probe failing, i.e. the script got *past* the guard. The last row is **R1**.

B7 — doctor exit codes, checked directly (no pipe)

Against synthetic indexes built in a throwaway `LUMEN_STORE` (one with 50 identical unit-norm 768-dim vectors among 100, one clean, one absent):

```
$ LUMEN_STORE="$SB/store" bash scripts/lumen-index-doctor.sh "$SB/
corruptproj"
```

```
index: .../sb7/store/proj-corrupt/index.db
model: ordis/jina-embeddings-v2-base-code
integrity_check: ok
files: 60 fully indexed, 1 queued placeholders | chunks: 100
vectors: 100 total, 51 distinct
   1 duplicate-vector group(s); 50 vectors (50.00%) are not unique
   largest identical group: 50 copies of ONE vector
per-vector: 0 NaN/Inf, 0 all-zero, 0 off-norm

   CORRUPTION DETECTED

$ LUMEN_STORE="$SB/store" bash scripts/lumen-index-doctor.sh "$SB/
corruptproj" >/dev/null 2>&1; echo $?
1
$ LUMEN_STORE="$SB/store" bash scripts/lumen-index-doctor.sh "$SB/
healthyproj" >/dev/null 2>&1; echo $?
0
$ LUMEN_STORE="$SB/store" bash scripts/lumen-index-doctor.sh "$SB/
noindexproj" >/dev/null 2>&1; echo $?
2
```

The duplicate group was invisible to every per-vector test (`0 NaN/Inf, 0 all-zero, 0 off-norm`) — exactly the failure mode the header describes. The *"never opens the DB for writing"* claim also holds: with the DB at mode `444` inside a mode-`555` directory it still returns 0, and leaves no `-wal`/`-shm`/journal side files.

Caveat, by design but worth stating: the threshold is `groups[0] >= 10`, so a stale-vector group of 2-9 copies is reported in the output but does **not** set the failure exit code.

C8 — the ACTION REQUIRED detection adapts, in both directions

Driven through `SETUP_WIZARD_LIB_ONLY=1` with the detection block extracted **verbatim** from `scripts/setup-agents-wizard.sh:1249-1260`, under `env -i` with a throwaway `$HOME`, `$CLAUDE_CONFIG_DIR` and `$PROJECT_ROOT`:

`$CLAUDE_CONFIG_DIR` state	**project genome**	**manual steps emitted**
`plugins/cache/ashlr-marketplace/ashlr` only	—	**1** — *"Activate the ashlr plugin inside Claude Code"*
	absent	**1** — *"Initialise the ashlr genome for this*

`$CLAUDE_CONFIG_DIR` state	project genome	manual steps emitted
`+ plugins/marketplaces/ashlr-marketplace`		*project*" (activation step **gone**)
`+ plugins/marketplaces/ashlr-marketplace`	present	**0**

```
################ DIRECTION 1 ################
CLAUDE_DIR resolved to: .../sb8/cfg
MANUAL_TITLES count: 1
  TITLE: Activate the ashlr plugin inside Claude Code

################ DIRECTION 2: now ALSO create plugins/
marketplaces/ashlr-marketplace ################
MANUAL_TITLES count: 1
  TITLE: Initialise the ashlr genome for this project (optional,
improves routing)

################ DIRECTION 2b: marketplaces present AND project
genome present ################
MANUAL_TITLES count: 0
```

The distinction the comment claims — *cache = the installer cloned it; marketplaces = the operator actually activated it* — is real and correctly implemented.

C9 — `MANUAL-STEPS.md` written and gitignored

```
$ ls -la MANUAL-STEPS.md
-rw-r--r-- 1 milosvasic milosvasic 913 Aug 27 17:34 MANUAL-
STEPS.md
$ git check-ignore -v MANUAL-STEPS.md
.gitignore:72:MANUAL-STEPS.md   MANUAL-STEPS.md
$ git status --porcelain --ignored MANUAL-STEPS.md
!! MANUAL-STEPS.md
```

Content is real, generated (`Generated 2026-08-27T15:34:23Z by scripts/setup-agents-wizard.sh`), 5 steps, each with a runnable command block.

C10 — bounded *in that section*; the wizard as a whole is not

Every probe **inside the ACTION REQUIRED section** is bounded:

- `scripts/setup-agents-wizard.sh:1263` — `timeout 20 journalctl …` whose `--since` is itself `$(timeout 10 systemctl show …)`
- `scripts/setup-agents-wizard.sh:1296` — `timeout 20 lumen search "x" -p … --summary -n 1`

The section-scoped claim is therefore **true**. The broader grep you asked for is **not**:

line	invocation	bounded?
552	`journalctl -u ollama --no-pager -o cat --since "-30 days"`	**NO** (measured 2.31 s here; unbounded by construction)
870	`claude mcp get lumen`	**NO**
872	`claude mcp add-json lumen …`	**NO**
1076	`lumen index "$PROJECT_ROOT" 2>&1 | tail -3`	**NO — a full index run with no timeout**
1119	`claude mcp get lumen`	**NO**
991, 1146	`timeout 25 mimo mcp list`	yes
1263, 1296	`timeout 20 journalctl` / `timeout 20 lumen search`	yes

Line 1076 is the one that matters: `lumen index` on this project has been running for **8 h 9 min** at the time of writing. It is gated behind `WIZARD_INDEX_PROJECT` (test A25), so it is opt-in — but when opted in, the wizard has no upper bound at all.

The guarding test is narrower than its name suggests:

```
unbounded=$(printf '%s\n' "$src_code" | grep -cE '(^|[^a-
        z0-9_-])lumen search ' | head -1)
bounded=$(printf '%s\n' "$src_code" | grep -cE 'timeout [0-9]+
        lumen search ')
assert_eq "A47 every 'lumen search' probe in the wizard is
        timeout-bounded" "$unbounded" "$bounded"
```

✅ It covers `lumen search` **only** — not `lumen index`, not `journalctl`, not `claude mcp`. (The trailing `| head -1` on a `grep -c` is a no-op.)

D11 — test suite run (three times — the repo moved mid-pass, see the note below)

At HEAD `88ab20c` (the commit this verification was commissioned against)

```
$ bash scripts/test-setup-agents-wizard.sh --no-live
REAL_EXIT_CODE=0
  total=126 passed=119 failed=0 skipped=7
```


```
{"timestamp_utc":"20260827T160219Z",
 "host":{"uname":"Linux/x86_64","bash":"5.2.37(1)-release"},
 "totals":{"total":126,"passed":119,"failed":0,"skipped":7},
 "exit_code":0}
```


At HEAD `72dc135` (after a concurrent actor pushed mid-pass)

```
$ bash scripts/test-setup-agents-wizard.sh --no-live
REAL_EXIT_CODE=0
  total=134 passed=127 failed=0 skipped=7
```

Current working tree (after the scripts were patched mid-pass)

```
$ bash scripts/test-setup-agents-wizard.sh --no-live
REAL_EXIT_CODE=0
  total=141 passed=134 failed=0 skipped=7
```

Group I grew from 14 to 21 assertions (I15-I21, added in response to R1/R2/R4). All green. `~/.bashrc`, `~/.bash_profile`, `~/.local/bin/lumen` and `~/.claude.json` were re-hashed around this run too — identical.

Groups: 10 — A B C D E F G H I J — emitted in the order A, B, C, D, E, F, **H, G**, I, J (cosmetic: H is printed before G).

group		n
A	Static analysis of the wizard	54
B		8

group		n
	Lumen launcher unit tests (isolated `$HOME`)	
C	Shell configuration unit tests (isolated `$HOME`)	10
D	MCP configuration unit tests (isolated `$HOME`)	5
E	SuperSpec submodule safety (data-loss regression)	3
F	Live integration (real environment)	7 — **all 7 skipped** by `--no-live`
G	Backup manifest and rollback (isolated `$HOME`)	18
H	Telemetry opt-out (isolated `$HOME`)	7
I	Operational scripts (remediation / reindex / doctor)	14
J	Hardcoded-path audit *(new in `72dc135`)*	8

Failures: 0 in both runs. The 7 skips are exactly F1-F7, the live-integration group, in both runs. Group J (J1-J8) is entirely new and entirely green.

The repository moved underneath this verification

At the start of this pass HEAD was `88ab20c`. Partway through, a **concurrent actor** committed and pushed `72dc135 "Remove all hardcoded machine paths (18 files, 33 occurrences) + audit guard"` — and swept this in-progress report file into that commit. **No git command in this pass was mutating**; the commit was not mine.

I re-checked what that commit touched:

```
$ git diff 88ab20c 72dc135 --stat -- scripts/
 scripts/audit-hardcoded-paths.sh    | 109 ++++++++++++++++++++++++++
```

```
++++++++++++
 scripts/test-setup-agents-wizard.sh |  84 ++++++++++++++++++++++++++++++
++++

$ git diff 88ab20c 72dc135 --stat -- scripts/setup-agents-
wizard.sh \
      scripts/lumen-reindex.sh scripts/lumen-index-doctor.sh \
      scripts/ollama-vulkan-remediation.sh AGENTS.md CLAUDE.md
GEMINI.md QWEN.md
(no output — unchanged)
```

The wizard, the three operational scripts and the four carriers were NOT touched. Every finding in this report — R1, R2, R3, R4, and items 4–10, 12, 15 — therefore still applies verbatim at current HEAD. Only the test totals in this item moved (126 → 134), and item 14's commit hash.

D12 — three untold assertions mutation-tested (all load-bearing) — plus one worthless assertion found

Method: the whole `scripts/` tree was copied to a throwaway project root. The copy reproduced the baseline exactly (`total=126 passed=119 failed=0 skipped=7`). One mutation was applied at a time to the **copy**; the real wizard was never touched.

1. `B3 launcher selects highest version (sort -V, not lexical)` — group B

Mutation at line 280: `sort -V -k1,1` → `sort -k1,1`.

```
< done | sort -V -k1,1 | tail -n1 | cut -f2
> done | sort -k1,1 | tail -n1 | cut -f2
(mutant still parses)
     [57] B3 launcher selects highest version (sort -V, not
lexical)
  total=126 passed=118 failed=1 skipped=7
```

Behavioural: the test plants 0.0.9 / 0.0.41 / 0.0.100 launchers and executes the wrapper, asserting it prints `0.0.100`. Lexical sort picks `0.0.9`. **Load-bearing.**

2. `C7 PATH guard written literally ($PATH not expanded at install)` — group C

Mutation at line 348: `case ":\$PATH:"` → `case ":$PATH:"` (expands at install time).

```
< case ":\$PATH:" in *":\$HOME/.local/bin:"*) ;; *) export PATH="\
$HOME/.local/bin:\$PATH";; esac
> case ":$PATH:"  in *":\$HOME/.local/bin:"*) ;; *) export PATH="\
$HOME/.local/bin:\$PATH";; esac
(mutant still parses)
     [69] C7 PATH guard written literally ($PATH not expanded at
install)
     [70] C8 sourcing .bashrc adds ~/.local/bin exactly once
  total=126 passed=117 failed=2 skipped=7
```

Two assertions caught it. **Load-bearing.**

3. `E1 REGRESSION: gitlink submodule is NOT deleted` — group E

Mutation: `if [[ -e "$sub_path/.git" ]]` → `if [[ -d "$sub_path/.git" ]]` — reintroducing the historical data-loss bug, since a checked-out submodule's `.git` is a **file**.

```
< if [[ -e "$sub_path/.git" ]]; then
> if [[ -d "$sub_path/.git" ]]; then
(mutant still parses)
     [6]  A6 submodule presence test uses -e (gitlink is a FILE)
     [78] E1 REGRESSION: gitlink submodule is NOT deleted
     [79] E2 gitlink submodule reported as a submodule
  total=126 passed=116 failed=3 skipped=7
```

E1 builds a real git repo with a gitlink `.git` file plus a canary `IMPORTANT.txt`, runs `setup_superspec`, and asserts the canary `SURVIVED`. Under the mutation the canary is `rm -rf`'d. **Load-bearing, and the strongest assertion in the suite.**

After restoring the pristine copy: `total=126 passed=119 failed=0 skipped=7`.

Worthless assertions found among the three mutation-tested: none.

But a fourth one is worthless — see R5. After the scripts were patched mid-pass I checked the newly added group-I assertions and found `I21 doctor maps unexpected failures to 2, not 1` green on provably broken behaviour, and still green with the guard it names deleted outright.

Related weaknesses, noted without mutation testing:

- `A48`/`A49`/`A50` and `I16`-`I21` are bare `assert_contains` greps over script source. They catch deletion of a literal; they assert nothing about behaviour.
- `A47` is narrower than its name — it covers `lumen search` only (see C10).
- `I11 reindex refuses to start on a Vulkan backend` greps for the refusal branch without ever exercising the detector. It was green throughout the entire lifetime of **R1**, which is exactly how R1 shipped.
- `I15` is the counter-example worth copying: it *runs* the script and asserts an exit code.

The suite's ratio matters here: **54 of 141 assertions (38%) are group A static source greps**, and much of groups I and J are the same shape. A suite can be 141-green and still miss a live defect in the thing it is testing, as R3 and R5 together demonstrate.

D13 — the real environment is untouched by a `--no-live` run

```
=== BEFORE ===
45dfceaf29b60c49da965895ee152a0ae2b4890ed18a8cfbb0940dfe2470f877
<home>/.bashrc
5affcade3a9c2fe707518aabd7c635be9231f7d68845be94aca5daa6ecc1849b
<home>/.bash_profile
ebf11baf1e6c90fccc8c630a32fd0a23d91cd190918f2f7d7e2a18918229ae7c
<home>/.local/bin/lumen
52caecc09cf656a1b45bbd6e17f3cf07414157a556ef4b35541646038f9f6d21
<home>/.claude.json

=== AFTER ===
45dfceaf29b60c49da965895ee152a0ae2b4890ed18a8cfbb0940dfe2470f877
<home>/.bashrc
5affcade3a9c2fe707518aabd7c635be9231f7d68845be94aca5daa6ecc1849b
<home>/.bash_profile
ebf11baf1e6c90fccc8c630a32fd0a23d91cd190918f2f7d7e2a18918229ae7c
<home>/.local/bin/lumen
52caecc09cf656a1b45bbd6e17f3cf07414157a556ef4b35541646038f9f6d21
```

```
<home>/.claude.json

=== DIFF ===
IDENTICAL - real environment untouched
```

The run's only side effect is `.test-evidence/<stamp>/`, which is gitignored (`.gitignore:63`).

E14 — HEAD == remote for the root and all five carriers

Checked twice, because the root moved mid-pass (see D11). Equal both times.

```
=== ROOT, first check ===
local HEAD : 88ab20c7c2b0f27db2d869148035a66b74c11e10
branch     : main
ls-remote  : 88ab20c7c2b0f27db2d869148035a66b74c11e10   refs/
heads/main

=== ROOT, re-check after the concurrent push ===
$ git rev-parse HEAD
72dc135e89a6879e37379bfed90c9b2366010ae8
$ git ls-remote origin refs/heads/main
72dc135e89a6879e37379bfed90c9b2366010ae8    refs/heads/main
```

submodule	local HEAD	superproject gitlink	ls-remote refs/heads/main
`ai_interviewing`	`ed73d8558e289ca0254b4ccc45e0df810767d3ae`	same	same
`design-toolkit`	`efd2c3fb2f880aaf2baf7f4819ce28a1ce3609cb`	same	same
`milosvasic.ru`	`66c8d607b20f0d9e984cb0e06f10a2020ebd9a10`	same	same
`monetization`	`54ed7b0f5add52821d18866facb5ee8c75adef69`	same	same
`vasic.digital`	`6e5411c21b3cd9c6df5addb543b1a07930c3bfa9`	same	same

All six are on `main`; local HEAD, the superproject's recorded gitlink, and the remote tip agree in every case.

Re-verified after the concurrent push: the four carriers are still byte-identical from line 24 (`961f4713…`), and the five submodule gitlinks are unchanged by `72dc135`.

Note — the working tree went dirty during this verification pass, then was committed by someone else. Git status was reported clean at the start of this session; by 18:02 it read:

```
 M _tools/deploy-langs.sh
 M docs/setup-agents-wizard/OLLAMA-REMEDIATION.md
 M docs/setup-agents-wizard/README.md
 M milosvasic.ru
 M vasic.digital
?? .ashlrcode/
?? _tools/deploy-langs.sh.bak.20260827180158
?? docs/setup-agents-wizard/ACTION-REQUIRED.md
?? docs/setup-agents-wizard/OPERATIONAL-SCRIPTS.md
```

None of it was mine — this pass created only this report. Another actor was writing to the repo concurrently (the `.bak.20260827180158` suffix timestamps it inside this session's window), and subsequently committed and pushed it all as `72dc135`. The two `M` submodule entries were dirty *content* (`sitemap.xml` in each), not moved pointers; the gitlinks are unchanged, which is why E14 holds at both hashes. **"HEAD equals remote" is true; "the repository is quiescent" is a different statement, and was false throughout this pass.**

E15 — the four root carriers share one byte-identical body

```
$ for f in AGENTS.md CLAUDE.md GEMINI.md QWEN.md; do
      printf '%s  %s\n' "$(tail -n +24 "$f" | sha256sum | cut -d'
' -f1)" "$f"; done
961f47134720080e69eafc5f7458702ad2af91dbaa7eb279f33e9d2f417851c9
AGENTS.md
961f47134720080e69eafc5f7458702ad2af91dbaa7eb279f33e9d2f417851c9
CLAUDE.md
961f47134720080e69eafc5f7458702ad2af91dbaa7eb279f33e9d2f417851c9
GEMINI.md
961f47134720080e69eafc5f7458702ad2af91dbaa7eb279f33e9d2f417851c9
QWEN.md

distinct body hashes: 1
```

Whole-file hashes differ (`8bd411ea…`, `e8b8511f…`, `96404cfc…`, `c49127da…`) — the only divergence is the 23-line per-tool header (tool name, inherited-from path, audience line). All four are at their committed state (`git status --porcelain` on them is empty).

Recommended fixes, in priority order

#	Where	Fix	Why now
~~1~~	`scripts/lumen-reindex.sh` `backend_library()`	~~scan from `ActiveEnterTimestamp`~~	**R1 — DONE mid-pass**, re-verified: `<EMPTY>` → `cpu`
~~2~~	`scripts/lumen-reindex.sh` arg loop	~~add `--help|-h` and reject unknown flags~~	**R2 — DONE mid-pass**, re-verified: `--help` exits 0 without indexing, `--forse` exits 2
3	`scripts/lumen-index-doctor.sh` python body	Wrap it in `try/except Exception` → print reason → `sys.exit(2)`. **The `case $rc in 0|1|2)` guard added mid-pass does not do this** — it forwards rc=1, which is exactly what an unhandled python exception produces	**R3, STILL LIVE** — an uninspectable index reports as *corrupt* and triggers a needless full rebuild
~~3b~~	`scripts/ollama-vulkan-remediation.sh --help`	~~`sed -n '2,34p'`~~	**R4 — DONE mid-pass**, re-verified: no source leakage
4	`scripts/test-setup-agents-wizard.sh` (I21)	Replace the `assert_contains "could not complete"` grep with a behavioural test: build a temp index DB with no `vec_chunks_vector_chunks00`, run the doctor, assert `rc == 2`	**R5, STILL LIVE** — I21 is green today on broken behaviour and green with the guard deleted
5	`scripts/test-setup-agents-wizard.sh` (I11, I16-I20)	Make them *execute* the scripts with stubs instead of grepping source	I11 was green for the entire lifetime of **R1**; a behavioural

#	Where	Fix	Why now
			version would have caught it
6	`scripts/setup-agents-wizard.sh:1076`	Bound `lumen index` (e.g. `timeout "${WIZARD_INDEX_TIMEOUT:-3600}"`) and widen A47 beyond `lumen search`	An unbounded index inside the wizard; the run on this host is at 8 h and counting
7	`scripts/setup-agents-wizard.sh:552, 870, 872, 1119`	Wrap the 30-day `journalctl` and the `claude mcp` calls in `timeout`	Same class as 6, lower blast radius

Method notes / reproducibility

- Sandboxes: `env -i` with a throwaway `$HOME`, `$CLAUDE_CONFIG_DIR`, `$LUMEN_STORE`, `$LUMEN_REINDEX_LOG` and `$PROJECT_ROOT`. `lumen`, `curl`, `ollama` and `journalctl` were replaced by sentinel-writing stubs whenever a script under test could otherwise have reached the real system.
- Mutation testing ran against a **copy** of `scripts/` under a throwaway project root. The copy reproduced the baseline exactly before any mutation, and the pristine file was restored and re-verified after the last one.
- The detection block in item 8 was extracted byte-for-byte from the wizard (`sed -n '1249,1260p'`, sha256 `74ead23e533a874ab19949d9e45ed70f1bed7fc7be1a1a5c239e12edcd0a4e60`) rather than re-typed, so what was tested is what ships.
- `/etc/sysconfig/ollama`, `~/.bashrc`, `~/.bash_profile`, `~/.local/bin/lumen`, `~/.claude.json` and the ollama unit's `InvocationID` were all hashed/recorded before and after every step that could conceivably have touched them. All identical afterwards.

Final state at the close of this pass

Report closed	2026-08-27 ~18:40 CEST

Root HEAD	`72dc135` == `origin/main`
Working tree	`scripts/lumen-reindex.sh`, `lumen-index-doctor.sh`, `ollama-vulkan-remediation.sh`, `test-setup-agents-wizard.sh` modified **by a concurrent actor, not by this pass**
Test suite	141 assertions, 134 pass, 0 fail, 7 skip (`--no-live`), exit 0
ollama	`active (running)` since 09:44:18, `NRestarts=0`, `library=cpu`, 0 i915 faults since start
Live defects	**R3** (doctor exit-code contract) and **R5** (worthless assertion `I21`)
Open question	**A3** — the 32-distinct-vector probe, still unanswered by the backend

---

Nothing in the real environment was modified by this pass. `/etc/sysconfig/ollama` (`713ad8ca…`), `~/.bashrc` (`45dfceaf…`), `~/.bash_profile` (`5affcade…`), `~/.local/bin/lumen` (`ebf11baf…`) and `~/.claude.json` (`52caecc0…`) are byte-identical to their values at the start, and the ollama unit's `InvocationID` (`5b04994a…`) is unchanged.